Clang AST Passes

CS24BTECH11030, CS24BTECH11036

October 2026

Contributions. CS24BTECH11030: Part 1 (AST dump and node analysis) and the traversal mechanisms section. CS24BTECH11036: Part 2 (the three AST-matcher passes, their tests and write-up).

1 Part 1: Exploring AST dumps

1.1 Program

```
int f(int x, int& y) {
  return x * x + y * y;
}

int main() {
  int k = 10, sum = 0;

  for (int i = 0; i < k; i++) {
    for (int j = 0; j < k; j++) {
      sum += f(i, j);
    }
  }

  int cnt = 0;
  while (sum != 1) {
    if (sum % 2) {
      sum = 3 * sum + 1;
    } else {
      sum /= 2;
    }

    cnt++;
  }

  return cnt;
}
```

1.2 Dumping the AST

```
clang -Xclang -ast-dump -fno-color-diagnostics -fsyntax-only nested_loops.cpp > ast_dump.txt
```

Running the above command dumps the AST into a local txt file. Below is a reduced version of the AST dump of `nested_loops.cpp`

```
TranslationUnitDecl 0x6239bf35e318 <<invalid sloc>> <invalid sloc>
|-TypedefDecl 0x6239bf35eb88 <<invalid sloc>> <invalid sloc> implicit __int128_t '__int128'
```

```
| `-BuiltinType 0x6239bf35e8e0 '__int128'
| ...
|-FunctionDecl 0x6239bf3a7eb8 <nested_loops.cpp:1:1, line:3:1> line:1:5 used f 'int (int, int &)'
| ...
`-FunctionDecl 0x6239bf3a8178 <line:5:1, line:26:1> line:5:5 main 'int ()'
  `-CompoundStmt 0x6239bf3c1ee8 <col:12, line:26:1>
    | ...
    |-ForStmt 0x6239bf3a88b8 <line:8:3, line:12:3>
    | |-DeclStmt 0x6239bf3a8460 <line:8:8, col:17>
    | | `-VarDecl 0x6239bf3a83d8 <col:8, col:16> col:12 used i 'int' cinit
    | |   `-IntegerLiteral 0x6239bf3a8440 <col:16> 'int' 0
    | |-<<<NULL>>>
    | |-BinaryOperator 0x6239bf3a84e8 <col:19, col:23> 'bool' '<'
    | | |-ImplicitCastExpr 0x6239bf3a84b8 <col:19> 'int' <LValueToRValue>
    | | | `-DeclRefExpr 0x6239bf3a8478 <col:19> 'int' lvalue Var 0x6239bf3a83d8 'i' 'int'
    | | `-ImplicitCastExpr 0x6239bf3a84d0 <col:23> 'int' <LValueToRValue>
    | |   `-DeclRefExpr 0x6239bf3a8498 <col:23> 'int' lvalue Var 0x6239bf3a8268 'k' 'int'
    | |-UnaryOperator 0x6239bf3a8528 <col:26, col:27> 'int' postfix '++'
    | | `-DeclRefExpr 0x6239bf3a8508 <col:26> 'int' lvalue Var 0x6239bf3a83d8 'i' 'int'
    | `-CompoundStmt 0x6239bf3a88a0 <col:31, line:12:3>
    |   `-ForStmt 0x6239bf3a8868 <line:9:5, line:11:5>
    |     |-DeclStmt 0x6239bf3a85e0 <line:9:10, col:19>
    |     | `-VarDecl 0x6239bf3a8558 <col:10, col:18> col:14 used j 'int' cinit
    |     |   `-IntegerLiteral 0x6239bf3a85c0 <col:18> 'int' 0
    |     |-<<<NULL>>>
    |     |-BinaryOperator 0x6239bf3a8668 <col:21, col:25> 'bool' '<'
    |     | |-ImplicitCastExpr 0x6239bf3a8638 <col:21> 'int' <LValueToRValue>
    |     | | `-DeclRefExpr 0x6239bf3a85f8 <col:21> 'int' lvalue Var 0x6239bf3a8558 'j' 'int'
    |     | `-ImplicitCastExpr 0x6239bf3a8650 <col:25> 'int' <LValueToRValue>
    |     |   `-DeclRefExpr 0x6239bf3a8618 <col:25> 'int' lvalue Var 0x6239bf3a8268 'k' 'int'
    |     |-UnaryOperator 0x6239bf3a86a8 <col:28, col:29> 'int' postfix '++'
    |     | `-DeclRefExpr 0x6239bf3a8688 <col:28> 'int' lvalue Var 0x6239bf3a8558 'j' 'int'
    |     `-CompoundStmt 0x6239bf3a8850 <col:33, line:11:5>
    |       `-CompoundAssignOperator 0x6239bf3a8820 <line:10:7, col:20> 'int' lvalue '+=' ComputeLHSTy='int'
              ComputeResultTy='int'
    |         |-DeclRefExpr 0x6239bf3a86c0 <col:7> 'int' lvalue Var 0x6239bf3a8308 'sum' 'int'
    |         `-CallExpr 0x6239bf3a87d8 <col:14, col:20> 'int'
    |           |-ImplicitCastExpr 0x6239bf3a87c0 <col:14> 'int (*)(int, int &)' <FunctionToPointerDecay>
    |           | `-DeclRefExpr 0x6239bf3a8768 <col:14> 'int (int, int &)' lvalue Function 0x6239bf3a7eb8 'f'
            'int (int, int &)'
    |           |-ImplicitCastExpr 0x6239bf3a8808 <col:16> 'int' <LValueToRValue>
    |           | `-DeclRefExpr 0x6239bf3a8728 <col:16> 'int' lvalue Var 0x6239bf3a83d8 'i' 'int'
    |           `-DeclRefExpr 0x6239bf3a8748 <col:19> 'int' lvalue Var 0x6239bf3a8558 'j' 'int'
    | ...
    |-WhileStmt 0x6239bf3c1e80 <line:15:3, line:23:3>
    | |-BinaryOperator 0x6239bf3a8a00 <line:15:10, col:17> 'bool' '!='
    | | |-ImplicitCastExpr 0x6239bf3a89e8 <col:10> 'int' <LValueToRValue>
    | | | `-DeclRefExpr 0x6239bf3a89a8 <col:10> 'int' lvalue Var 0x6239bf3a8308 'sum' 'int'
    | | `-IntegerLiteral 0x6239bf3a89c8 <col:17> 'int' 1
    | `-CompoundStmt 0x6239bf3c1e60 <col:20, line:23:3>
    |   |-IfStmt 0x6239bf3c1df8 <line:16:5, line:20:5> has_else
    |   | |-ImplicitCastExpr 0x6239bf3a8a98 <line:16:9, col:15> 'bool' <IntegralToBoolean>
    |   | | `-BinaryOperator 0x6239bf3a8a78 <col:9, col:15> 'int' '%'
    |   | |   |-ImplicitCastExpr 0x6239bf3a8a60 <col:9> 'int' <LValueToRValue>
    |   | |   | `-DeclRefExpr 0x6239bf3a8a20 <col:9> 'int' lvalue Var 0x6239bf3a8308 'sum' 'int'
    |   | |   `-IntegerLiteral 0x6239bf3a8a40 <col:15> 'int' 2
    |   | |-CompoundStmt 0x6239bf3a8ba8 <col:18, line:18:5>
    |   | | `-BinaryOperator 0x6239bf3a8b88 <line:17:7, col:23> 'int' lvalue '='
    |   | |   ...
    |   | `-CompoundStmt 0x6239bf3c1de0 <line:18:12, line:20:5>
    |   |   `-CompoundAssignOperator 0x6239bf3c1db0 <line:19:7, col:14> 'int' lvalue '/=' ComputeLHSTy='int'
              ComputeResultTy='int'
    |   |     ...
    |   `-UnaryOperator 0x6239bf3c1e48 <line:22:5, col:8> 'int' postfix '++'
    |     `-DeclRefExpr 0x6239bf3c1e28 <col:5> 'int' lvalue Var 0x6239bf3a8908 'cnt' 'int'
    `-ReturnStmt 0x6239bf3c1ed8 <line:25:3, col:10>
      `-ImplicitCastExpr 0x6239bf3c1ec0 <col:10> 'int' <LValueToRValue>
        `-DeclRefExpr 0x6239bf3c1ea0 <col:10> 'int' lvalue Var 0x6239bf3a8908 'cnt' 'int'
```

1.3 Interesting nodes and their hierarchy

1. `ForStmt`**.**

A `for` loop is one node with four fixed child slots, in order: *init* (`DeclStmt`), *condition variable* (here `<<<NULL>>>`, used only for `for (...; T v = ...; )` forms), *condition* (`BinaryOperator <`), *increment* (`UnaryOperator ++`), and finally the *body*. In the dump the body is a `CompoundStmt`, whose only child is the inner `ForStmt`. This is how loop nesting appears in the tree: the inner loop is a child of the outer loop's body.

2. `CallExpr`**.**

`f(i, j)` is denoted by `CallExpr` in the clang AST. Further, Its children are *function pointer* (Here achieved via `ImplicitCastExpr <FunctionToPointerDecay>`), and *parameters*. Another interesting thing which can be seen is the difference in how *CallByValue* and *CallByReference* affects the parameters passes. *CallbyValue* requires an `ImplicitCastExpr <LValueToRValue>`, while for *CallByReference*, we can directly pass in a reference `DeclRefExpr`.

3. `WhileStmt`**.**

A `while` loop has two child slots, in order: *condition* (Here a `BinaryOperator`), and *body* (`CompoundStmt`)

Other nodes worth noting.

`ImplicitCastExpr <LValueToRValue>` appears around every variable that is *read* (`i` and `k` in `i < k`) but not around `i` in `i++`, where it is modified in place.

`CompoundStmt` is the { ... } block and just holds a list of statements.

The overall root of this dump is the `TranslationUnitDecl`.

2 Traversal mechanisms in Clang

`RecursiveASTVisitor`**.** A CRTP base class. Derive from `RecursiveASTVisitor<Derived>` and define `VisitXxx(Xxx *node)` for the node kinds of interest; the base class walks the whole tree depth-first and calls them. Three layers exist per node kind: `TraverseXxx` (controls descent into children and can prune a subtree by not calling the base), `WalkUpFromXxx` (calls the `Visit` methods for the node and its base classes), and `VisitXxx` (what you usually override; return `false` to stop the whole traversal). Best for imperative analyses that need state across the walk.

AST matchers with `MatchFinder`**.** A declarative approach: describe the pattern as a composition of matchers (node matchers such as `forStmt`, `varDecl`; narrowing matchers such as `hasName`; traversal matchers such as `hasBody`, `hasCondition`, `hasDescendant`) and attach a `MatchCallback` to it with `MatchFinder::addMatcher`. `.bind("id")` labels sub-nodes, which the callback's `run` retrieves with `Result.Nodes.getNodeAs<T>("id")`. The finder does the walk internally. Concise for local patterns, and what Part 2 uses.

`ASTConsumer` **and** `ASTFrontendAction`**.** The hook by which a tool gets the AST at all. A `FrontendAction` creates an `ASTConsumer`, whose `HandleTranslationUnit(ASTContext&)` (or `HandleTopLevelDecl`) is called once parsing finishes. From there one typically starts a `RecursiveASTVisitor` from `Context.getTranslationUnitDecl()`. `newFrontendActionFactory(&Finder)` in a LibTooling `main` hides this for matchers.

`StmtVisitor / DeclVisitor`**.** Dispatch-only visitors: `Visit(node)` calls the `VisitXxx` for that node's dynamic type, but *does not* recurse; you recurse manually via `node->children()`. Used inside Clang itself (e.g. the AST printer, code generation) when the traversal order is custom.

Direct child iteration and parent queries. Every `Stmt` exposes `children()`, and a `DeclContext` exposes `decls()`, for a hand-written walk. Clang nodes have no parent pointer; `ASTContext::getParents(node)` (backed by the `ParentMapContext`) computes the parent map on demand for upward navigation.

3 Part 2: AST-matcher passes

All three matchers are in one file `pass.cpp`. Each check has its own test file in `tests/`. Every matcher only looks at the main file, so code from headers like `<stdio.h>` is ignored.

3.1 Build and run

The tool is built inside an LLVM 17.0.6 source tree. The target is called `MA4_Pass`, so it does not clash with the lab's `Simple_Pass`:

```
# run from the folder that contains 2-ast-matcher-passes/
cd path/to/CS24BTECH11030_CS24BTECH11036
cp -r 2-ast-matcher-passes ~/llvm-project/clang-tools-extra/ma4-passes
# add   add_subdirectory(ma4-passes)   to ~/llvm-project/clang-tools-extra/CMakeLists.txt
cd ~/llvm-project/build && cmake . && make -j4 MA4_Pass
bin/MA4_Pass ../clang-tools-extra/ma4-passes/tests/assign_in_condition.cpp --
```

`--` means "no compilation database".

Tested with LLVM/Clang 17.0.6.

3.2 Pass 1: assignment inside a condition

Goal. Find `if (x = 5)`, where `==` was probably meant.

Approach. We check the conditional statements inside `if`, `for`, `while`, `do-while` and `?:` constructs and in each, we look for an assignment operator `=`. The matcher for the `if` is shown below; the other four are the same with a different statement name, and `anyOf` joins all five.

```
ifStmt(hasCondition(ignoringParenImpCasts(binaryOperator(hasOperatorName("=")).bind("assignment"))))
```

In C++, `n = 13` is implicitly converted to bool, and `if ((n = 13))` has extra brackets. `ignoringParenImpCasts` skips extra nodes around the condition and without it, the above cases would be missed. We match only plain assignment statements `=`, not compound assignment operators like `+=`. The callback prints where the assignment is.

Test. `tests/assign_in_condition.cpp`

Output. The assignments on lines 5, 11, 17, 25, 29 are reported while the comparisons like `n == 10` on line 8 are not.

```
tests/assign_in_condition.cpp:5:8
tests/assign_in_condition.cpp:11:21
tests/assign_in_condition.cpp:17:11
tests/assign_in_condition.cpp:25:13
tests/assign_in_condition.cpp:29:10
```

3.3 Pass 2: empty loop body

Goal. Find `for`, `while` and `do-while` loops with an empty body {}.

Approach. For each of the three loops, we use `hasBody(compoundStmt(statementCountIs(0)))`. This matches statements having a body with 0 statements inside it. However, loops with a single statement as the body, or just a `;` (like `for (...);`), are not reported, as the question asks only for empty blocks.

Test. `tests/empty_loop_body.cpp`

Output. The empty `for` on line 13, the empty `do{}while()` on line 21 and the empty `while(){}` on line 25 are reported.

```
tests/empty_loop_body.cpp:13:5
tests/empty_loop_body.cpp:21:5
tests/empty_loop_body.cpp:25:5
```

3.4 Pass 3: unused local variable

Goal. Find local variables that are declared but never used.

Approach. We use two matchers decls and refs. decls records every local variable when it is declared. refs records every place a local variable is used. Each variable has one declaration node, and every use (referenced) points back to it, so we can tell variables apart even if they have the same name. We keep two sets: declared variables and referenced variables. After we process the whole file, any variable in the declared set but not in the referenced set is unused (unreferenced) and we print it. We must wait until the end because a variable may be used later in the file after it is declared. Function parameters, global variables and `static` locals are not reported. A variable that is only assigned to after declaration (`int c = 3; c = 4;`) also counts as used.

Test. `tests/unused_local.cpp`

Output. The unused `y` in `f`, unused `z` in `f` and the unused `x` in `main` are all reported. `n`, `i` and `x` in `f` are used, so they are not reported.

```
Unused variable: y
Unused variable: z
Unused variable: x
```

4 Limitations

- Pass 1 finds only statements with `=`. It misses `if (x += 2)`. It also misses assignments inside bigger conditions, like `if ((x = 1) && y)`, because we only check the whole condition and not break it into subparts.
- Pass 2 does not report `for (...);` or `while (...);`, or a body that only holds an empty inner block like `for(...){if(...){}}`.
- Pass 3 misses variables that are only assigned and never read. It does not report parameters, globals or `static` local variables. It prints only the name, not the line number of the variable which can be confusing if the variable name is used across multiple scopes.
- Code in headers is ignored, so problems in included files are not reported.

Note on AI use

AI assistance was used to organize and polish the report. The passes themselves were written by us.